

Lecture 7: Lists



INFO 1100: Introduction to Programming

We've learned several
ways of representing data

We've learned several
ways of representing data

Types!

What if we want to
combine and order data?

What if we want to
combine and **order** data?

Morning routine?

Steps in a recipe?

Songs at a concert?

Lists



Lists

a *sequence* of values

Lists

a *sequence* of values

each value is an *element*

Lists

[5, 6, 5, 9, 15, 23]

Lists

```
[ "Big", "Red" ]
```

Lists

```
[1 + 2, "Moby Dick", True]
```

Lists

```
[1 + 2, "Moby Dick", True]
```

lists can be **heterogeneous**
store different kinds of data

Lists

`[0, [1, 2, 3], 4]`

lists can be **nested**
contain other lists

Lists

Enter in the chat:

When might we want a nested list?

`[0, [1, 2, 3], 4]`

lists can be **nested**
contain other lists

Lists

`[]`

lists can be **empty**

len ()

`len()`

returns the # of elements in
the list

the same as for strings!

len()

len([5, 6, 5, 9, 15, 23])

len()

len([5, 6, 5, 9, 15, 23])



6

len()

len([1 + 2, "Moby Dick", True])

len()

len([1 + 2, "Moby Dick", True])



3

len()

len([0, [1, 2, 3], 4])

len()

len([0, [1, 2, 3], 4])



3

len()

len([])

len ()

len ([])



0

Lists are (also) **indexed**

Lists are (also) **indexed**

Every list element is at an **index**

Lists are (also) **indexed**

[5,	6,	5,	9,	15,	23]
0	1	2	3	4	5

Lists are (also) **indexed**

```
[1 + 2, "Moby Dick", True]  
  0      1      2
```

Lists are (also) **indexed**

```
x = [5, 6, 5, 9, 15, 23]
```

```
x[0] → 5
```

Lists are (also) **indexed**

```
x = [5, 6, 5, 9, 15, 23]
```

```
x[6] → IndexError
```

Lists are (also) **indexed**

```
x = [5, 6, 5, 9, 15, 23]
```

```
type(x[0]) → <class 'int'>
```


Lists are (also) **sliced**

Lists are (also) **sliced**

```
x = [5, 6, 5, 9, 15, 23]
```

```
x[0:3] → [5, 6, 5]
```

Lists are (also) **sliced**

```
x = [5, 6, 5, 9, 15, 23]
```

```
x[3:] → [9, 15, 23]
```

Lists are (also) **sliced**

```
x = [5, 6, 5, 9, 15, 23]
```

Slicing creates a **new** list

Lists are (also) **sliced**

```
x = [5, 6, 5, 9, 15, 23]
```

```
type(x[3:]) → <class 'list'>
```

Updating list elements

Updating list elements

Lists are **mutable**!

Updating list elements

Use **assignment**

Updating list elements

```
x = [ 'Wake', 'Brush', 'Class' ]
```

Updating list elements

```
x = [ 'Wake', 'Brush', 'Class' ]
```

```
x[2] = 'Gym'
```

Updating list elements

```
x = [ 'Wake', 'Brush', 'Class' ]
```

```
x[2] = 'Gym'
```

```
x → [ 'Wake', 'Brush', 'Gym' ]
```

Updating list elements

```
x = [ 'Wake', 'Brush', 'Class' ]
```

```
x[2] = 'Gym'
```

```
x → [ 'Wake', 'Brush', 'Gym' ]
```

still **the same list!**

We can also...

We can also...

- find
- add
- add many
 - reorder
 - remove

We can also...

- find
- add
- add many
 - reorder
 - remove

... list elements

find

<code>item in lst</code>	Whether <code>item</code> is in <code>lst</code> .
<code>lst.index(item)</code>	Index of the first occurrence of <code>item</code> is in <code>lst</code> .

find

<code>item in lst</code>	Whether <code>item</code> is in <code>lst</code> .
<code>lst.index(item)</code>	Index of the first occurrence of <code>item</code> is in <code>lst</code> .

add

<code>lst.insert(i, item)</code>	Insert <code>item</code> into <code>lst</code> just before the element at <code>i</code> , shifting all others
<code>lst.append(item)</code>	Insert <code>item</code> at the end of <code>lst</code> .

find

<code>item in lst</code>	Whether <code>item</code> is in <code>lst</code> .
<code>lst.index(item)</code>	Index of the first occurrence of <code>item</code> is in <code>lst</code> .

add

<code>lst.insert(i, item)</code>	Insert <code>item</code> into <code>lst</code> just before the element at <code>i</code> , shifting all others
<code>lst.append(item)</code>	Insert <code>item</code> at the end of <code>lst</code> .

add many

<code>lst1 + lst2</code>	Creates a new list containing items of <code>lst1</code> followed by items of <code>lst2</code> .
<code>lst1.extend(lst2)</code>	Append all items in <code>lst2</code> to <code>lst1</code> .

reorder

<code>lst.sort()</code>	Rearrange items of <code>lst</code> in ascending order.
<code>lst.reverse()</code>	Rearrange items of <code>lst</code> in reverse order.

reorder

<code>lst.sort()</code>	Rearrange items of <code>lst</code> in ascending order.
<code>lst.reverse()</code>	Rearrange items of <code>lst</code> in reverse order.

remove

<code>lst.remove(item)</code>	Remove first occurrence of <code>item</code> from <code>lst</code> . <code>ValueError</code> if <code>item</code> not in <code>lst</code> .
<code>lst.pop(i)</code>	Return and remove item at index <code>i</code> in <code>lst</code> . <code>IndexError</code> if <code>i</code> is not an index in <code>lst</code> .

Lists and **strings**

Lists and **strings**

Sometimes we want to convert
strings to lists and vice versa!

Lists and **strings**

Sometimes we want to convert
strings to lists and vice versa!

... why?

Lists and **strings**

```
s = "Emma"
```


Lists and **strings**

```
s = "Emma"
```

```
list(s) → ['E', 'm', 'm', 'a']
```

Lists and **strings**

```
s = "Pride and Prejudice"
```

Lists and **strings**

```
s = "Pride and Prejudice"
```

```
s.split() → ['Pride', 'and', 'Prejudice']
```

Lists and **strings**

```
s = "Pride and Prejudice"
```

```
s.split() → ['Pride', 'and', 'Prejudice']
```

Why might this be helpful?

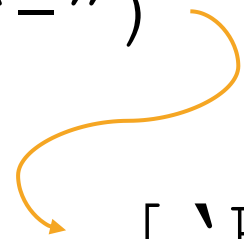
Lists and **strings**

```
s = "Pride-and-Prejudice"
```

Lists and **strings**

```
s = "Pride-and-Prejudice"
```

```
s.split("-")
```



```
['Pride', 'and', 'Prejudice']
```

Lists and **strings**

```
lst = ["Northanger", "Abbey"]
```

```
" ".join(lst) → "Northanger Abbey"
```

Lists **in memory**

Lists **in memory**

```
p1 = ["It", "is", "a", "truth"]
```

```
p2 = ["It", "is", "a", "truth"]
```

Lists **in memory**

```
p1 = ["It", "is", "a", "truth"]
```

```
p2 = ["It", "is", "a", "truth"]
```

$p1 \rightarrow \text{["It", "is", "a", "truth"]}$

$p2 \rightarrow \text{["It", "is", "a", "truth"]}$

Lists **in memory**

```
p1 = ["It", "is", "a", "truth"]
```

```
p2 = p1
```

Lists **in memory**

```
p1 = ["It", "is", "a", "truth"]
```

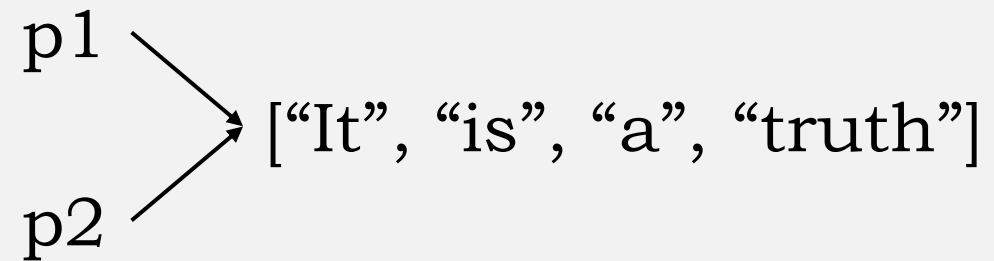
```
p2 = p1
```

aliasing

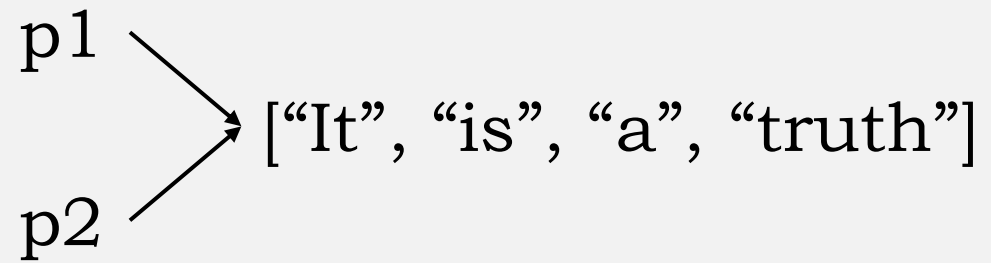
Lists **in memory**

```
p1 = ["It", "is", "a", "truth"]
```

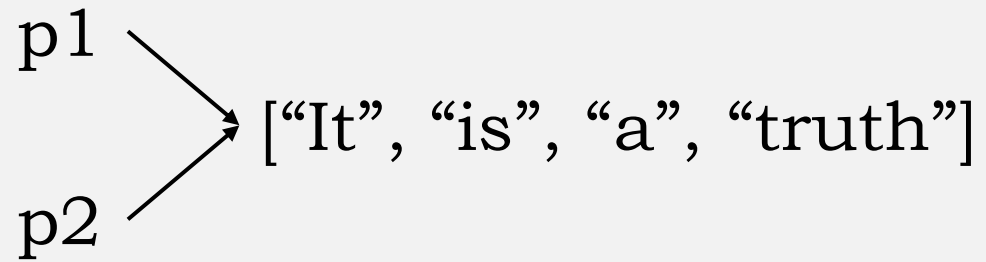
```
p2 = p1
```



Lists **in memory**



Lists **in memory**

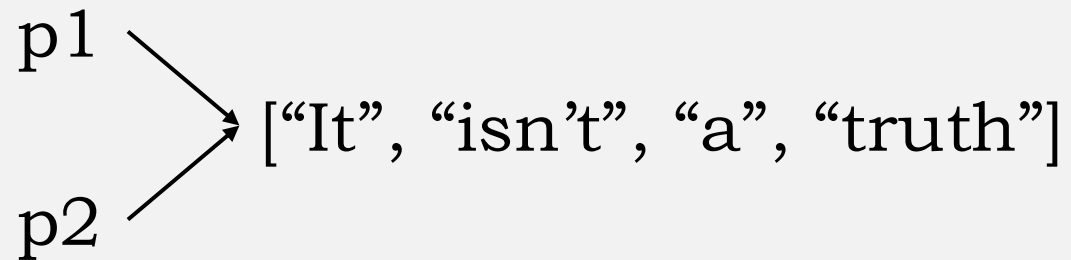


```
p2[1] = "isn't"
```

Lists **in memory**



```
p2[1] = "isn't"
```

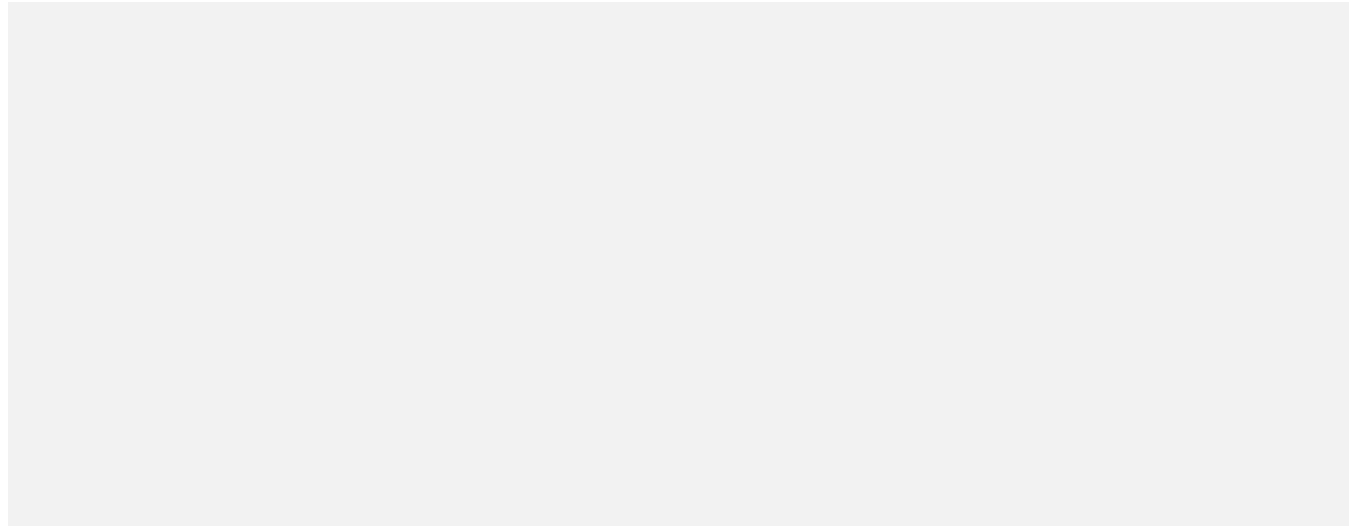


Lists **in memory**

```
def edit_list(lst):  
    lst.append("universally")  
  
p1 = ["It", "is", "a", "truth"]  
edit_list(p1)
```

Lists **in memory**

```
def edit_list(lst):  
    lst.append("universally")  
  
p1 = ["It", "is", "a", "truth"]  
edit_list(p1)
```



Lists **in memory**

```
def edit_list(lst):  
    lst.append("universally")
```

```
p1 = ["It", "is", "a", "truth"]
```

```
edit_list(p1)
```

$p1 \longrightarrow ["It", "is", "a", "truth"]$

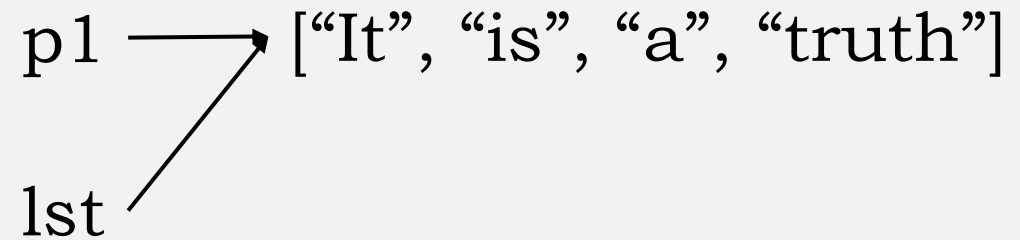
Lists **in memory**

```
def edit_list(lst):  
    lst.append("universally")  
  
p1 = ["It", "is", "a", "truth"]  
edit_list(p1)
```

$p1 \longrightarrow ["It", "is", "a", "truth"]$

Lists **in memory**

```
def edit_list(lst):  
    lst.append("universally")  
  
p1 = ["It", "is", "a", "truth"]  
  
edit_list(p1)
```



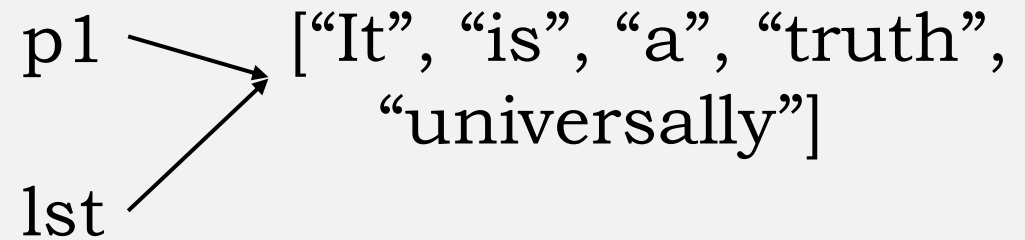
Lists **in memory**

```
def edit_list(lst):
```

```
    lst.append("universally")
```

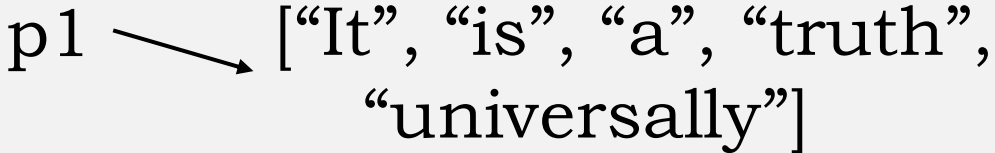
```
p1 = ["It", "is", "a", "truth"]
```

```
edit_list(p1)
```



Lists **in memory**

```
def edit_list(lst):  
    lst.append("universally")  
  
p1 = ["It", "is", "a", "truth"]  
edit_list(p1)
```



p1 → ["It", "is", "a", "truth",
"universally"]

Example

```
def mystery(l, n):  
    if n not in l and not max(l) < n:  
        l.extend([n])  
        l.sort()
```

```
vals = [1, 2, 3, 4, 5]
```

```
mystery(vals, 10)
```

```
mystery(vals, 7)
```


Example

```
def mystery(l, n):  
    if n not in l and not max(l) < n:  
        l.extend([n])  
        l.sort()
```

```
vals = [1, 2, 3, 4, 5]  
mystery(vals, 10)  
mystery(vals, 7)
```

**This is was one of the
quiz question options
from this week!**

Questions?

(... I'll wait for 3)

For tomorrow:

(07/03 @ 11:30am)

- Lab 7
- Quiz tomorrow!

For tomorrow:

(07/03 @ 11:30am)

- Lab 7
- Quiz tomorrow!

Monday:

(07/06 @ 9:00am)

- Assignment 2

Tuesday:

(07/07 @ 11:30am)

- Readings + GRQs